

Отчёт по Graph Coloring

Ломаченко Олег, ФПМИ ПМФ, 3 курс

1 Постановка задачи

В данной работе рассматривалась задача раскраски графа. На вход подаётся неориентированный граф

$$G = (V, E),$$

где V — множество вершин, а E — множество рёбер. Необходимо назначить каждой вершине $v \in V$ цвет $c(v)$ так, чтобы для любого ребра $(u, v) \in E$ выполнялось условие

$$c(u) \neq c(v).$$

Целевая функция — минимизировать число использованных цветов

2 Базовые эвристики с лекций: DSATUR и RLF

В первую очередь были реализованы и протестированы две эвристики, обсуждавшиеся на лекциях:

- DSATUR;
- RLF.

2.1 Результат базовых эвристик

Качество решений оказалось ограниченным:

- слабые пороги были достигнуты;
- ни один сильный порог достигнут не был.

При этом DSATUR в моих экспериментах работал лучше, чем RLF. Поэтому дальнейшие улучшения были построены именно вокруг DSATUR.

3 Попытка branch-and-bound

После базовых эвристик я попробовал пойти в другую сторону и реализовать branch-and-bound. Эта идея была вдохновлена тем, что branch-and-bound хорошо проявлял себя в задаче о рюкзаке.

Однако для задачи раскраски графа ситуация хуже. В задаче о рюкзаке есть естественная и достаточно сильная оценка сверху/снизу через линейную релаксацию, в отличие от покраски где оценки с низу тяжелые и слабые.

3.1 Нижняя оценка через LP-релаксацию

В качестве эксперимента я всё равно попробовал использовать нижнюю оценку, основанную на значении линейной программы для задачи раскраски (релаксация ILP модели с лекции). Для решения LP использовался внешний LP-solver.

3.2 Результат branch-and-bound

На практике эта идея показала себя плохо. LP-релаксация оказалась слишком слабой для этой задачи.

В результате branch-and-bound:

- работал медленно;
- давал результаты хуже, чем эвристики;
- не был пригоден для больших тестов.

Поэтому от этой идеи я отказался и вернулся к улучшению эвристических алгоритмов.

4 Улучшение DSATUR: перебор стартовых вершин

Я подумал над DSATUR и понял что он чертовски зависит от стартовой вершины, которая у меня выбиралась при прочих равных лексикографически.

Поэтому была реализована следующая модификация:

- каждая вершина графа по очереди выбирается как стартовая;
- эта вершина красится первым цветом;
- затем запускается DSATUR;
- из всех полученных раскрасок выбирается лучшая.

5 Рандомизация разрешения равенств в DSATUR

В DSATUR часто возникают ситуации, когда несколько вершин имеют одинаковую saturation degree и одинаковую обычную степень. В классическом детерминированном варианте требуется какое-то правило разрешения таких равенств, например выбор вершины с меньшим номером.

Я решил использовать это место как источник рандомизации. Основное правило DSATUR осталось прежним:

- 1) выбрать вершину с максимальной saturation degree;
- 2) при равенстве выбрать вершину с максимальной степенью.

Но если после этих двух критериев всё ещё остаётся несколько кандидатов, то вершина выбирается случайно равновероятно среди них.

Так как алгоритм стал рандомизированным, для каждой стартовой вершины было сделано несколько запусков. В первых версиях я использовал около 20 запусков на каждую стартовую вершину. Такая версия уже дала заметный результат: удалось взять несколько сильных порогов.

В финальной версии число запусков на каждую стартовую вершину было увеличено до

$$\text{runs_per_start} = 40.$$

6 Локальное улучшение: сжатие цветовых классов

После того как DSATUR строит раскраску, получается разбиение вершин на цветовые классы:

$$C_0, C_1, \dots, C_{k-1},$$

где каждый C_i является независимым множеством.

Я заметил, что если некоторый цветовой класс очень маленький, то часто его можно полностью расформировать. Интуитивно на достаточно случайных графах трудно ожидать, что каждая вершина маленького класса имеет соседей во всех остальных цветовых классах. Значит, может оказаться возможным перекрасить вершины этого класса в другие цвета.

6.1 Удаление маленького цветового класса

Алгоритм рассматривает цветовые классы в порядке возрастания размера. Для очередного маленького класса C_i проверяется, можно ли для каждой вершины $v \in C_i$ найти другой допустимый цвет.

Цвет c считается допустимым для вершины v , если среди соседей v нет вершин цвета c :

$$\forall u \in N(v) : c(u) \neq c.$$

Если для всех вершин класса C_i удалось найти допустимые новые цвета, то класс C_i полностью удаляется, а число цветов уменьшается на единицу.

6.2 Выбор цвета при перекрашивании

Если вершину можно перекрасить в несколько разных цветов, то выбирается цвет того класса, который сейчас имеет наибольший размер.

То есть при перекрашивании маленького класса я стараюсь вливать его вершины в наиболее крупные допустимые классы. Это было сделано из следующей логики: если усиливать большие классы, то маленькие классы остаются маленькими, и их, возможно, получится удалить на следующих шагах.

6.3 Повторение процедуры

После удаления одного класса раскраска меняется. Поэтому процедура запускается снова:

- снова сортируются классы по размеру;

- снова ищется маленький класс, который можно расформировать;
- если такой класс найден, он удаляется;
- если ни один класс удалить нельзя, процедура останавливается.

Это локальное улучшение применялось после каждого построенного решения DSATUR.

7 Усиленное сжатие: понижение цветов отдельных вершин

После экспериментов стало понятно, что простого удаления маленьких классов недостаточно. Иногда нельзя сразу удалить ни один цветовой класс, но можно изменить структуру раскраски так, чтобы маленькие классы стали ещё меньше, а крупные — ещё крупнее.

Для этого была добавлена дополнительная фаза понижения цветов.

7.1 Переупорядочивание цветов по размеру классов

Сначала цветовые классы сортируются по убыванию размера. После этого самому большому классу присваивается цвет 0, следующему по размеру — цвет 1, и так далее.

То есть маленький номер цвета начинает означать большой цветовой класс.

7.2 Понижение цвета вершины

Далее алгоритм идёт по цветовым классам в порядке убывания размера, начиная со второго класса. Для каждой вершины текущего класса проверяется, можно ли перенести её в цвет с меньшим номером.

Если вершина v имеет цвет $c(v)$, то ищется минимальный допустимый цвет

$$c' < c(v),$$

такой что среди соседей v нет цвета c' .

Если такой цвет существует, вершина перекрашивается в него.

Эта операция сама по себе не всегда уменьшает число цветов. Но она уплотняет крупные классы и может сделать некоторые малые классы ещё меньше. После этого снова запускается процедура удаления маленьких классов.

7.3 Итоговая процедура

Финальная процедура локального улучшения устроена так:

- 1) удалить маленькие цветовые классы до упора;
- 2) переупорядочить классы по убыванию размера;
- 3) попытаться понизить цвета отдельных вершин;

- 4) снова удалить маленькие цветовые классы;
- 5) если число цветов уменьшилось, повторить процедуру.

Такой постпроцессинг оказался полезным, потому что DSATUR строит хорошую раскраску, но не пытается явно удалить уже созданные лишние цветовые классы.

С применением сжатий у меня осталось только два теста на которых не достигался сложный порог - это тест на 250 и на 1000

8 Фаза улучшения лучшего найденного решения

После базовой фазы DSATUR и сжатия была добавлена отдельная фаза улучшения текущего лучшего решения с ограничением по времени.

Идея состоит в том, чтобы не строить каждую раскраску заново, а частично сохранять структуру уже найденного хорошего решения.

8.1 Фиксация крупных цветовых классов

Пусть текущее лучшее решение использует k цветов. Его цветовые классы сортируются по убыванию размера:

$$C_0, C_1, \dots, C_{k-1}.$$

Затем выбирается некоторое число первых классов:

$$C_0, C_1, \dots, C_{t-1},$$

которые фиксируются. Все вершины этих классов сохраняют свои цвета, а остальные вершины считаются непокрашенными.

После этого оставшаяся часть графа достраивается заново.

8.2 Выбор числа фиксируемых классов

Число фиксируемых классов выбирается случайно по геометрическому распределению. Параметр распределения подбирается так, чтобы математическое ожидание было порядка

$$\sqrt{k},$$

где k — число цветов в текущем лучшем решении.

Такой выбор был сделан просто по вайбу.

Возможно, более правильным было бы выбирать число фиксируемых классов умнее, например искать переход от больших цветовых классов к маленьким. Но в финальной реализации я использовал более простую схему с геометрическим распределением.

После фиксации первых t крупных цветовых классов оставшиеся вершины достраиваются не обычным DSATUR, а рандомизированной версии DSATUR.

На каждой итерации выбирается случайное значение

$$\alpha \in [0.9, 1.0].$$

Затем:

- с вероятностью α следующая вершина выбирается по правилу DSATUR;
- с вероятностью $1 - \alpha$ следующая вершина выбирается случайно равновероятно среди всех непокрашенных вершин.

После выбора вершина красится в минимальный допустимый цвет.

Смысл этой процедуры в том, что алгоритм почти всегда ведёт себя как DSATUR, но иногда делает случайный ход. Это позволяет получать более разнообразные траектории и выходить из неудачных локальных структур.

9 Финальная версия алгоритма

Итоговая версия состоит из двух больших фаз.

9.1 Первая фаза: построение начального лучшего решения

На первой фазе:

- 1) перебираются все стартовые вершины;
- 2) для каждой стартовой вершины выполняется несколько случайных запусков DSATUR;
- 3) в финальной версии использовалось 40 запусков на каждую стартовую вершину;
- 4) после каждого запуска применяется squeeze;
- 5) лучшее решение сохраняется.

9.2 Вторая фаза: улучшение лучшего решения

На второй фазе:

- 1) берётся текущее лучшее решение;
- 2) его цветовые классы сортируются по убыванию размера;
- 3) случайно выбирается число фиксируемых крупных классов;
- 4) остальные вершины перекрашиваются рандомизированным DSATUR;
- 5) если число цветов уменьшилось, решение обновляется.

Эта фаза работает до достижения тайм-лимита.

Финальные параметры, которые показали лучший результат в моих экспериментах:

`runs_per_start = 40,`

`improvement_time_limit_sec = 240.`

10 Результаты

В финальной версии удалось достичь всех слабых порогов. Также удалось достичь всех сильных порогов, кроме самого большого теста. Для теста `gs_1000_5` лучшее найденное значение составило

109

цветов, тогда как сильный порог для него равен 100.